

ONNX-Scala

Relazione PPS
A.A. 2023/2024

Serafini Andrea

Gennaio 2025

Indice

Abstract	i
1 Introduzione	1
1.1 Scala e Type Safety	1
1.1.1 Type Safety	1
1.1.2 Type Safety nei modelli di AI	2
1.2 Interoperabilità nei sistemi di Machine Learning	3
2 ONNX-Scala	4
2.1 ONNX	4
2.1.1 Che cos'è ONNX	4
2.1.2 Architettura e design del formato ONNX	5
2.2 La libreria ONNX-Scala	7
3 Progetti correlati	9
3.1 Type Safety	9
3.1.1 Neurocat	9
3.1.1.1 Confronto tra ONNX-Scala e Neurocat	9
3.1.2 Tf-dotty / TensorFlow	10
3.2 Cross-platform	10
3.2.1 sbt-crossproject	10
3.2.2 Scala.js	11
3.2.3 Scala Native	12
3.2.4 Cats	13
3.2.4.1 Cats Effect	13
4 Applicazione e test	15
4.1 SqueezeNet	15
4.2 Demo	15
4.2.1 Full-model API	15
4.2.2 Fine-grained API	20
5 Risultati e conclusioni	22
5.1 Benchmark	22
5.2 Conclusioni	23

Elenco delle figure

1.1	Type Safety [1]	2
2.1	Ruolo di ONNX [2]	5
2.2	Logo ONNX-scala	7
3.1	Logo di Neurocat	9
3.2	Logo di ScalaJS	11
3.3	Logo e motto di Scala Native [3]	12
3.4	Logo Cats [4]	13
4.1	Foto di gatto	16
4.2	Rete neurale feed-forward	20
5.1	Risultati dei benchmark	22

Elenco dei sorgenti

1	Esempio d'uso: tf-dotty	10
2	Esempio d'uso: sbt-crossproject	11
3	Esempio d'uso: Scala.js	12
4	Python: Funzione di inferenza	16
5	Python: Preprocessing	16
6	Python: Risultato postprocessing	17
7	Scala: Preprocessing	17
8	Scala: Inferenza	18
9	ONNX-Scala: Inferenza	19
10	Rete neurale feed-forward	21
11	Scala: Fine-grained API	21
12	Codice JUnit per l'esecuzione dei test	22

Abstract

Il lavoro di ricerca descritto in questa relazione esplora l'integrazione tra il linguaggio Scala e il formato ONNX (*Open Neural Network Exchange*), con l'obiettivo di migliorare l'interoperabilità e la sicurezza dei tipi nei sistemi di Machine Learning. Per questo motivo si analizzerà la libreria ONNX-Scala, progettata per consentire l'uso del formato ONNX nell'ecosistema Scala, offrendo un'interfaccia tipizzata per l'inferenza, la modifica e la gestione di modelli di apprendimento automatico, a diversi livelli di applicazione.

L'analisi si concentrerà sui vantaggi dell'uso di Scala, tra cui il controllo statico dei tipi e l'integrazione con tecnologie come ONNX Runtime, confrontandolo con strumenti alternativi come Neurocat e Tf-dotty. La ricerca include test applicativi basati sul modello SqueezeNet e confronta diverse implementazioni, utilizzando Python, Scala puro e ONNX-Scala, evidenziando prestazioni e scalabilità. Infine, saranno riportati i risultati dei benchmark a dimostrazione di come ONNX-Scala possa migliorare l'efficienza in scenari di inferenza multipla, confermando la sua utilità per applicazioni distribuite e sistemi complessi.

Capitolo 1

Introduzione

Lo scopo di questa parte della ricerca è quello di esplorare soluzioni per l'impiego di tecnologie approfondite nel corso di Paradigmi di Programmazione e Sviluppo e l'interoperabilità tra sistemi di *Machine Learning* (ML). Verranno analizzati i principali vantaggi derivanti dall'uso di Scala in combinazione con strumenti di Machine Learning, evidenziando le proprietà chiave del linguaggio e dei sistemi coinvolti.

In questo primo capitolo si vedrà il concetto di *Type Safety* e le sue implicazioni nell'ambito dell'apprendimento automatico. Dopodiché verrà analizzato cosa si intende per *interoperabilità* e che proprietà ne stanno alla base.

1.1 Scala e Type Safety

Scala è un linguaggio fortemente tipizzato, il che significa che i tipi di dati vengono verificati in fase di compilazione, riducendo il rischio di errori a runtime. Questo si contrappone a linguaggi come Python, ampiamente utilizzati in ambito di apprendimento automatico, che tuttavia utilizzano tipizzazione dinamica e possono introdurre errori difficili da rilevare in fase di sviluppo. Considerando contesti come quelli del Machine Learning, dove operazioni tra strutture dati complesse, principalmente matrici e tensori, sono fortemente dispendiose in termini di tempo e di capacità di calcolo richiesta, gli aspetti legati alla Type Safety risultano fondamentali in quanto un errore a runtime potrebbe risultare molto oneroso da gestire.

1.1.1 Type Safety

In informatica con Type Safety, sicurezza rispetto ai tipi, o Type Soundness, robustezza dei tipi, ci si riferisce alla capacità di un linguaggio di programmazione di prevenire o disincentivare errori legati ai tipi di dato, pertanto questi concetti sono fondamentali per garantire l'affidabilità e la robustezza del codice, riducendo il rischio di bug e comportamenti imprevisti durante l'esecuzione.

Errori di tipo si possono verificare in vari scenari, quando si tenta ad esempio di eseguire operazioni su valori di tipi non compatibili, come sommare una stringa e un intero senza una gestione esplicita di tale caso.

Con controllo dei tipi si indica invece l'operazione con cui si determina se il valore assegnato a una variabile sia ammissibile o meno, in base al suo tipo. Esso può avvenire a compiletime o a runtime:

- se effettuato a compiletime, viene prodotto un elaborato già sottoposto alle verifiche e quindi non saranno necessari controlli a runtime. Ciò permette un'esecuzione più veloce del programma, a fronte di controlli preliminari più stringenti,
- se effettuato a runtime, il codice prodotto subisce meno controlli, che vengono messi in atto solo se una porzione di codice viene eseguita, ciò comporta che nel caso un ramo dell'esecuzione del programma contenente un errore di tipo non venisse mai visitato durante la fase di test a causa di una coverage non totale, questo potrebbe nascondere un errore che emergerebbe solamente nella fase di utilizzo del programma.

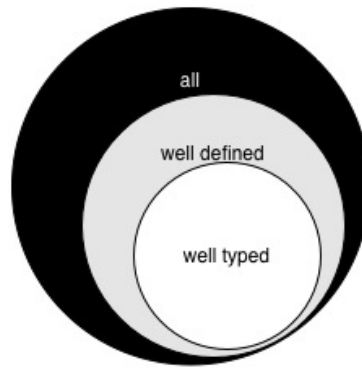


Figura 1.1: Type Safety [1]

Il concetto di Type Safety è legato a quello di Memory Safety, proprietà che limita la possibilità di trasferire arbitrariamente bit da una posizione di memoria a un'altra dando vita a errori di vario genere. Per questo motivo si ritiene fondamentale che un linguaggio offra meccanismi automatici per la gestione della memoria, come il Garbage Collector o altri meccanismi che eseguono controlli a compiletime.

I linguaggi di programmazione, riguardo alla Type Safety, possono essere classificati in:

- **Linguaggi non sicuri:** come C e C++, consentono esplicitamente errori di tipo attraverso operazioni come il cast o l'uso di *dangling pointers*, puntatori che si riferiscono ad aree di memoria non più valide, poiché già liberate o poiché il puntatore viene utilizzato all'esterno del contesto di esistenza della variabile cui si riferisce.
- **Linguaggi quasi sicuri:** effettuano la maggior parte dei controlli, ma come per la categoria precedente trascurano alcuni aspetti, come i *dangling pointers*.
- **Linguaggi sicuri:** come Java e Scala, garantiscono un controllo completo sui tipi richiesti.

L'essere un linguaggio sicuro implica quindi vincoli più rigorosi e una maggiore affidabilità nella scrittura del codice, superando inoltre le limitazioni del controllo a runtime che talvolta prevedono una ridotta efficienza dovuta all'aumento dei controlli durante l'esecuzione e l'impossibilità di gestire direttamente l'allocazione della memoria e la rimozione degli oggetti.

1.1.2 Type Safety nei modelli di AI

I modelli di Machine Learning si basano su tensori, che sono strutture dati multidimensionali.

Ogni tensore è caratterizzato da un tipo di dato associato, ad esempio `float32`, `int64` o `bool`, una forma che ne indica la dimensione in ogni asse, rappresentata come una tupla (`dim1`, `dim2`, ..., `dimN`), e un numero di dimensioni, anche detto Rank.

In questo contesto, la Type Safety aiuta a prevenire errori derivanti da inconsistenze nei dati e nelle pipeline di elaborazione assicurando che le operazioni sui tensori, come somme, moltiplicazioni o convoluzioni, siano effettuate tra tipi compatibili. In aggiunta, la corretta definizione dei tipi dei dati di input e output è essenziale per garantire che il modello riceva i dati attesi e produca risultati validi. Ad esempio, un modello che richiede input di tipo `float32` potrebbe fallire se gli viene fornito un input `int32` senza conversione esplicita.

Il formato aperto ONNX che sarà trattato del capitolo successivo, si pone come obiettivo l'interoperabilità tra framework di Deep Learning come PyTorch, TensorFlow e altri, ponendo particolare attenzione alla Type Safety, definendo chiaramente i tipi dei tensori utilizzati in un modello, incluso il tipo di dato *dtype*, la dimensione del tensore stesso e il suo layout. Inoltre, prima di eseguire un modello ONNX, strumenti come `onnx.checker` verificano che i tipi definiti siano conformi alle specifiche. Questo impedisce errori a runtime durante l'inferenza, come il tentativo di sommare tensori con tipi incompatibili. Durante la conversione di modelli tra framework, come ad esempio nel passaggio da TensorFlow a PyTorch, la Type Safety di ONNX assicura che i tipi e le operazioni siano preservati o, se necessario, adattati.

1.2 Interoperabilità nei sistemi di Machine Learning

Con interoperabilità nell'ambito di sistemi di Machine Learning, ci si riferisce alla capacità di diversi strumenti, framework e piattaforme di lavorare insieme senza richiedere modifiche significative ai modelli, ai dati o alle infrastrutture, spesso molto costose. È un elemento fondamentale per migliorare la flessibilità, la scalabilità e la portabilità dei progetti di apprendimento automatico.

Le principali sfide nell'implementazione dell'interoperabilità riguardano:

- Differenze nei framework: ogni framework di Machine Learning, come PyTorch, TensorFlow o scikit-learn, ha le proprie convenzioni per operazioni, gestione dei layer e formati dei dati. Queste differenze impediscono il passaggio trasparente tra due framework, e possono complicare la conversione dei modelli da un framework all'altro, rendendo necessario un lavoro di adattamento più oneroso.
- Perdita di precisione: durante la conversione dei modelli tra framework diversi è possibile che si verifichi una perdita di precisione o di dettaglio. Questo può compromettere l'accuratezza complessiva del modello, specialmente in quei domini in cui si rende necessaria la massima precisione dei dati.
- Prestazioni variabili: ottimizzare modelli su piattaforme diverse o su hardware specializzato potrebbe richiedere tuning di parametri specifici per garantire prestazioni ottimali.

Un esempio concreto di interoperabilità può essere quello di un sistema di analisi del comportamento utente che integra modelli sviluppati in PyTorch e TensorFlow. Utilizzando entrambi i framework in un ambiente scalabile, è possibile sfruttare i vantaggi distintivi di ciascuno: PyTorch per la flessibilità nel definire architetture neurali complesse e TensorFlow per la gestione ottimizzata di grandi dataset e operazioni su grafi.

Questo approccio non solo consente di migliorare l'efficienza complessiva del sistema, ma anche di combinare le capacità specializzate di diversi framework per ottenere risultati migliori e più robusti nell'analisi dei dati e nelle previsioni.

Capitolo 2

ONNX-Scala

In questo capitolo verrà prima approfondito uno dei formati aperti più adottati nel mondo dei modelli di apprendimento, che si pone come obiettivo quello di favorire l'interoperabilità nei sistemi di Machine Learning e Deep Learning: ONNX. Successivamente verrà poi analizzato come esso permetta, tramite la libreria ONNX-Scala, oggetto di questo progetto di ricerca, di sfruttare i vantaggi di modelli consolidati di Machine Learning con quelli offerti dall'ecosistema e dalle proprietà di Scala.

2.1 ONNX

ONNX, pronunciato *onyx*) come la pietra onice, è un progetto creato inizialmente da Microsoft e Facebook, che tutt'ora contribuiscono al mantenimento, portato avanti con il supporto non solo di giganti come Amazon AWS e Google ma anche di un'attiva comunità di appassionati che si pone come obiettivo quello di rendere interoperabili strumenti utilizzati nell'ambito dello sviluppo di applicazioni di Machine Learning e Deep Learning, favorendo la condivisione di modelli e superando le limitazioni dei singoli ecosistemi. Lo scopo dichiarato è quello di *democratizzare* lo sviluppo di applicazioni AI semplificandone il flusso di lavoro tramite la rimozione di barriere che gli sviluppatori potrebbero incontrare utilizzando strumenti eterogenei.

2.1.1 Che cos'è ONNX

ONNX, acronimo di *Open Neural Network eXchange*, è un formato di file aperto progettato per rappresentare tramite grafi di calcolo, modelli di apprendimento automatico in modo da poterli utilizzare in diversi framework e hardware di apprendimento automatico.[5]

Alcuni vantaggi derivanti dall'adozione di ONNX sono:

- **Interoperabilità software:** ONNX aderisce al formato **Protobuf**, abbreviazione di Protocol Buffers, un meccanismo *open source*, *language neutral*, *platform neutral* ed estensibile, sviluppato da Google per la serializzazione di dati strutturati[6]. Questo permette a ONNX di definire un insieme comune di operatori per consentire agli sviluppatori di utilizzare modelli con una varietà di framework, strumenti, runtime e compilatori, migliorando la collaborazione e riducendo i costi di sviluppo.
- **Interoperabilità hardware:** ONNX permette in molteplici scenari di superare il problema delle dipendenze legate all'hardware nei modelli di intelligenza artificiale, permettendo di utilizzare gli stessi modelli su diversi dispositivi con supporto per l'accelerazione hardware di CPU, GPU ed NPU. La computazione per i modelli ONNX è spesso distribuita su diversi processori grazie alla compatibilità con ONNX Runtime. Questo runtime migliora significativamente la velocità di inferenza e rende possibile l'implementazione di CPU in tempo reale. In più ONNX offre la possibilità di estendere l'elaborazione ai dispositivi edge e mobili, favorendo scenari di inferenza *on-device* e in tempo reale.

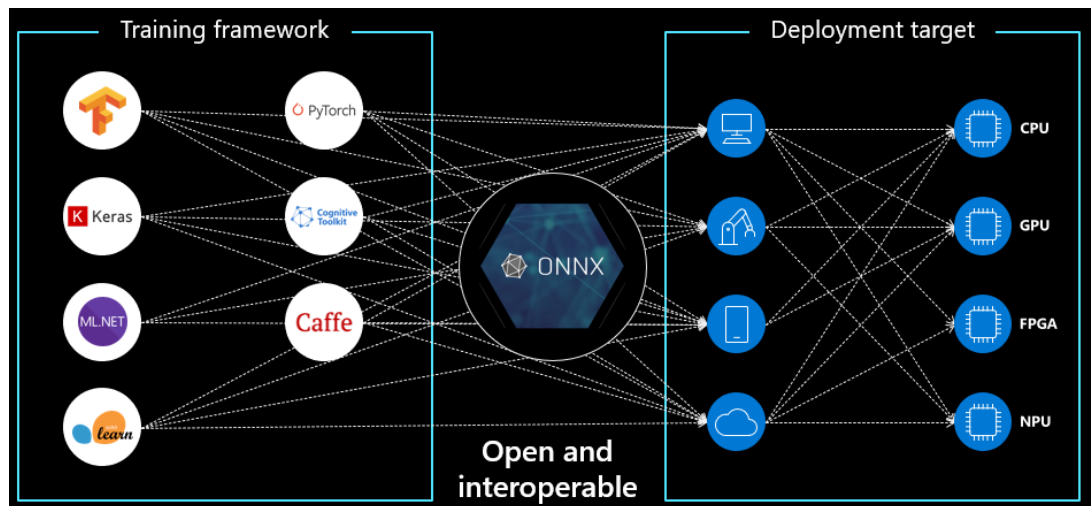


Figura 2.1: Ruolo di ONNX [2]

2.1.2 Architettura e design del formato ONNX

Il formato ONNX è basato su un approccio modulare che include:

- **Rappresentazione dei modelli basata su grafi:** un modello ONNX è rappresentato come un DAG, ovvero un grafo aciclico diretto. Questa struttura a grafo è un modo universale di rappresentare i modelli di apprendimento automatico, dove i nodi rappresentano le operazioni o le computazioni e gli archi rappresentano i flussi di dati tra di essi. Questo formato è facilmente adattabile a vari framework che adottano la stessa rappresentazione per i modelli.
- **Operatori standardizzati:** ONNX fornisce un set di operatori [7] che garantiscono compatibilità tra diversi framework. Sono disponibili ad esempio operatori per lavorare con convoluzioni, come `conv`, `convInteger` e `convTranspose`, normalizzazioni, come `InstanceNormalization`, `GroupNormalization` e `Local Response Normalization`, e funzioni di attivazione, come `relu`, `sigmoid` e `Leaky ReLU`. Quando un modello viene convertito nel formato ONNX, la sua architettura e i suoi pesi vengono tradotti in questa rappresentazione comune. Questa uniformità garantisce che il modello possa essere compreso da qualsiasi struttura che supporti ONNX.
- **Versioning e retrocompatibilità:** ONNX mantiene un sistema di versioning per i suoi operatori. Ciò garantisce che, anche se lo standard si evolve, i modelli creati nelle versioni precedenti rimangano utilizzabili. La retrocompatibilità è una caratteristica fondamentale che impedisce ai modelli di diventare rapidamente obsoleti in un ambiente che evolve continuamente.
- **Runtime ottimizzato:** ONNX si integra con ONNX Runtime, un motore di esecuzione altamente performante che sfrutta acceleratori hardware come GPU e TPU per migliorare l'efficienza.

ONNX propone inoltre dei meccanismi mirati a risolvere problemi specifici, come la *Type Denotation* [8] e la *Dimension Denotation* [9].

Questi meccanismi sono utili a descrivere informazioni semantiche riguardanti gli input e gli output. Per utilizzare ad esempio un modello per la classificazione di immagini come SqueezeNet l'utilizzatore ha necessità di conoscere e verificare numerose informazioni sui dati, come per esempio:

- l'input è una immagine
- l'immagine adotta il formato NCHW
- il modello di colore adottato è BGR
- un pixel utilizza 8bit
- il valore del pixel è normalizzato in un intervallo compreso tra 0 e 255

La Type Denotation permette, affiancata a Dimension Denotation e Model Metadata, di restituire tutte queste informazioni. Il funzionamento prevede che venga definito un insieme di tipi semantici che descrivono ciò che i modelli generalmente accettano in ingresso e restituiscono come risultato.

Per quanto riguarda la Type Denotation sono definite le seguenti etichette standard:

- **TENSOR**: indica che un tipo contiene un tensore generico
- **IMAGE**: indica che un tipo contiene un'immagine. È possibile utilizzare le denotazioni delle dimensioni per ottenere maggiori informazioni sul layout dell'immagine e anche le proprietà dei metadati opzionali del modello
- **AUDIO**: indica che un tipo contiene un file audio
- **TEXT**: indica che un tipo contiene del testo

Per utilizzare questa specifica sarà sufficiente agli sviluppatori aggiungere l'etichetta corretta ai dati utilizzati per garantire la correttezza del tipo in input e output.

La Dimension Denotation è invece utilizzata per etichettare tramite descrizioni semantiche e tipi di dato gli assi di un tensore, per poi poter effettuare verifiche su tali assi.

Si articola in tre fasi principali:

1. **Definizione delle etichette**: si stabilisce un insieme di tipi per i tensori, bilanciando la granularità per evitare errori e semplificare la comprensione agli utenti
2. **Propagazione delle etichette**: consiste nell'implementare meccanismi per trasmettere le informazioni semantiche attraverso le operazioni della rete neurale, garantendo che le etichette siano mantenute o aggiornate correttamente durante le trasformazioni dei tensori
3. **Verifica delle etichette**: permette di introdurre passaggi di verifica che utilizzano le etichette per identificare incongruenze o errori nel flusso dei dati, migliorando l'affidabilità e la correttezza dei modelli.

ONNX predispone anche un modo per inferire la forma dei tensori, chiamato appunto *Shape Inference*. Questo meccanismo analizza il grafo computazionale e propaga le informazioni sulle forme dei tensori attraverso i vari nodi, utilizzando regole predefinite per ciascun operatore. La Shape Inference è essenziale per l'ottimizzazione dei modelli, in quanto permette di verificare la correttezza della struttura del grafo e migliorarne l'efficienza computazionale. ONNX fornisce strumenti nativi per eseguire l'inferenza delle forme, come la funzione `onnx.shape_inference.infer_shapes`, che può essere utilizzata per validare e ottimizzare i modelli prima della loro esecuzione su dispositivi edge o in ambienti di produzione. Tuttavia la corretta inferenza delle forme può risultare complessa quando il modello include operatori dinamici o input con dimensioni variabili, richiedendo strategie avanzate per garantire compatibilità ed efficienza.

In generale, la flessibilità del design permette a ONNX di evolversi con aggiornamenti regolari e seguendo le proposte e le necessità degli utenti, aggiungendo nuovi operatori e funzionalità per rimanere al passo con le innovazioni nell'ambito dell'apprendimento automatico.

2.2 La libreria ONNX-Scala



Figura 2.2: Logo ONNX-scala

La libreria ONNX-Scala è una soluzione progettata per integrare il formato ONNX con l'ecosistema Scala, permettendo di interagire con i modelli di Deep Learning più avanzati. Questa libreria offre un'interfaccia tipizzata per fare inferenze su, caricare e modificare modelli ONNX, consentendo agli sviluppatori di sfruttare la sicurezza dei tipi e le caratteristiche funzionali di Scala per applicazioni di Machine Learning scalabili.

Alcune delle caratteristiche principali di ONNX-Scala includono:

- **Interfaccia funzionale:** consente agli sviluppatori di definire pipeline di inferenza in modo chiaro e conciso, sfruttando i concetti di immutabilità e composizione
- **Supporto per ONNX Runtime:** la libreria si basa su ONNX Runtime per eseguire modelli, garantendo alte prestazioni e compatibilità con gli operatori standardizzati
- **Integrazione con Scala 3:** sfrutta le funzionalità avanzate di Scala 3, come i singleton types, per fornire sicurezza durante la manipolazione di tensori e operazioni matematiche [10].

ONNX-Scala mette a disposizione dell'utente due API:

- **Full-model:** un'API per l'utilizzo *black-box* del modello
- **Fine-grained:** un'API più granulare per un maggior controllo sul modello

La prima API permette all'utente di passare un file `.onnx` direttamente al backend sottostante per caricare un modello già addestrato, permettendo così di effettuare un'ottimizzazione dell'intero grafo. Questo tipo di utilizzo garantisce una Type Safety minima, per cui l'utilizzatore dovrà preoccuparsi di definire correttamente i tensori utilizzati per le inferenze. Secondo i benchmark riportati dallo sviluppatore stesso della libreria, le performance riscontrate nelle inferenze effettuate attraverso queste API e utilizzando invece ONNX Runtime direttamente da Python, ovvero la modalità più utilizzata e diffusa, avrebbero uno scarto medio compreso nel range 3-10%.

La seconda API, definibile come di più basso livello, permette all'utente di modellare le specifiche di ONNX utilizzando un DSL Scala. Questa API viene utilizzata a differenza di quella precedente per effettuare pre e post preprocessing di tensori, personalizzazione dei modelli, streaming dei parametri interni al modello, trasferimento dell'apprendimento o ensembling. In questa API è poi presente una maggior Type Safety implementata per permettere di sviluppare programmi complessi senza perdere traccia dei dati manipolati. Utilizzando quindi questa API ogni operazione rappresentata da un nodo del grafo ONNX viene eseguita singolarmente dal backend, permettendo la definizione da zero di un modello in Scala utilizzando appunto le operazioni della libreria ONNX-Scala, iniettando i parametri da sorgenti esterne in base alle necessità. Sarà così possibile definire strutture dinamiche dei grafi nelle quali l'esecuzione stessa ne definirà la topologia, in maniera simile alla funzionalità offerta dalla recente *Eager Execution* di TensorFlow.

Si raggiunge quindi un compromesso tra flessibilità ed efficienza, andando a perdere la possibilità di eseguire un'ottimizzazione sul grafo completo e aggiungendo l'overhead a ogni istruzione richiamata, per questi motivi per quanto riguarda le performance in questo caso lo sviluppatore ha registrato un'esecuzione circa il 20% più veloce rispetto all'utilizzo di NumPy nel caso di applicazione su reti MLP completamente connesse di dimensioni ragionevoli, mentre si hanno esecuzioni fino a cinque volte più lente nel caso di reti come SqueezeNet che richiedono un numero elevato di piccole operazioni.

Entrambe le API sono supportate dal backend ottimizzato ONNX Runtime, che permette di sfruttare sia CPU che GPU. Attualmente i backend utilizzati sono due, uno basato come già detto su ONNX Runtime per quanto riguarda l'utilizzo attraverso la JVM, mentre il secondo utilizzato per Scala.js/JavaScript si basa su ONNX Runtime Web[11].

Alcuni dei vantaggi nell'impiego di ONNX-Scala riguardano pertanto:

- **Sicurezza dei tipi:** minimizza errori a runtime attraverso il controllo statico
- **Scalabilità:** ideale per applicazioni distribuite grazie alla compatibilità con Spark e altre librerie di Big Data.
- **Interoperabilità:** permette di integrare facilmente modelli addestrati in diversi framework

Un impiego concreto di ONNX-Scala è quello di utilizzare robusti *scoring service* scritti in Scala, basati quindi su JVM, su modelli allenati utilizzando vari framework scritti in Python, come PyTorch o TensorFlow.[12]

Capitolo 3

Progetti correlati

In questo capitolo verranno analizzate ulteriori tecnologie inerenti all'ambito dell'interoperabilità nei sistemi di Machine Learning, analizzando delle alternative a ONNX-Scala, come ad esempio Neurocat, e illustrandone pro e contro, per poi parlare di tecnologie ausiliarie che permettono di ampliare gli orizzonti dell'interoperabilità.

3.1 Type Safety

In seguito si illustreranno librerie che sono state fonte di ispirazione per la nascita di ONNX-Scala, che svolgono lo stesso compito e si focalizzano sul concetto di interoperabilità e Type Safety.

3.1.1 Neurocat

Neurocat è una libreria open source basata su Python che nasce nel 2017 dal desiderio di incrementare il livello di robustezza nel testing di applicazioni safety-critical, di fatto l'omonima azienda è specializzata in ambito automotive e può vantare Volkswagen tra i primi partner.



Figura 3.1: Logo di Neurocat

Questo modulo si integra bene con le più famose librerie di Machine Learning e Data Science come NumPy, PyTorch e TensorFlow, rendendola versatile per diverse applicazioni. Essa è disponibile su GitHub [13], ed è progettata per aiutare a costruire, manipolare e analizzare grafi etichettati, fornendo strumenti per lavorare con strutture dati complesse in ambito di Machine Learning e Deep Learning. È ampiamente utilizzata in ambito accademico e industriale per test e validazione di sistemi di apprendimento automatico.

Tra le funzionalità principali di Neurocat troviamo, in modo analogo a ONNX, la creazione e manipolazione di grafi atti a rappresentare le relazioni in un modello di Machine Learning, con etichette sui nodi e sugli archi. In seguito verranno quindi illustrate le differenze tra queste due tecnologie.

3.1.1.1 Confronto tra ONNX-Scala e Neurocat

Le librerie Neurocat e ONNX-Scala sono entrambe progettate per lavorare con modelli di Machine Learning, ma hanno scopi e funzionalità differenti.

Neurocat, ad esempio, nasce come libreria per il testing e la validazione di modelli, mentre ONNX-Scala è una libreria per l'utilizzo e la manipolazione di modelli ONNX specificatamente nell'ambiente Scala.

La più grande differenza tra le due librerie, tuttavia, è l'ecosistema per cui sono state concepite che vincola Neurocat ad ambienti Python e ONNX-Scala ad ambienti Scala (adottando il più generico formato ONNX che tuttavia supera le limitazioni del singolo ecosistema).

3.1.2 Tf-dotty / TensorFlow

Tf-dotty è una libreria sperimentale per Dotty, un compilatore per lo sviluppo su Scala 3 che permette di gestire i DOT, ovvero i Dependent Object Type, che mira a fornire un'interfaccia tipizzata per TensorFlow sfruttando le funzionalità avanzate di Scala 3, verificando staticamente i tipi e le dimensioni dei tensori.

L'integrazione tra TensorFlow e Scala è storicamente problematica a causa di due fattori principali:

- Mancanza di un supporto ufficiale da parte di Google per un'API idiomatica in Scala.
- Difficoltà nella gestione dei tensori e dei grafi computazionali all'interno di un linguaggio strettamente tipizzato.

Tf-dotty nasce con l'obiettivo di colmare questo divario, sfruttando le metaprogrammazioni avanzate di Scala 3 per garantire Type Safety e Shape Safety nella costruzione dei modelli e nella gestione dei tensori. Questo approccio riduce il rischio di errori a runtime, un problema comune nelle implementazioni Python di TensorFlow.

Tf-dotty utilizza un'architettura basata su Singleton Types e Shapeless, due tecniche avanzate di Scala che permettono di garantire la coerenza dei tipi durante la definizione delle operazioni sui tensori. Il sistema traccia la dimensione dei tensori attraverso tipi letterali e utilizza nuove funzionalità del *typechecker* per verificare le operazioni sui tensori durante la compilazione.[14] È possibile vederne un esempio riportato nel listato 1.

```
1 val tensor = tf.zeros(10 #: 20 #: 30 #: SNil)
2 // ...: Tensor[Float, 10 #: 20 #: 30 #: SNil]
3 tf.reduce_mean(tensor, axis = 0 :: SNil)
4 // ...: Tensor[Float, 20 #: 30 #: SNil]
5 tf.reduce_mean(tensor, axis = 1 :: 2 :: SNil)
6 // ...: Tensor[Float, 10 #: SNil]
```

Listing 1: Esempio d'uso: tf-dotty

Nell'esempio vediamo come venga definito un tensore tridimensionale, popolato di zeri, e il suo tipo, visibile nel commento sottostante. Si può poi osservare la forma del tensore cambiare dopo le operazioni di riduzione del tensore stesso, risultanti in una diminuzione degli assi.

In questo caso il compilatore è in grado di verificare se le operazioni sui tensori sono corrette senza bisogno di eseguire il codice, evitando errori di dimensione, come invece avverrebbe tentando di ridurre un asse inesistente. L'uso di tipi letterali, come #: e SNil, permette di rappresentare chiaramente la struttura di un tensore nel sistema di tipi di Scala.

3.2 Cross-platform

L'ecosistema Scala offre diverse soluzioni per la programmazione *cross-platform*, garantendo interoperabilità tra ambienti diversi, da JavaScript a sistemi nativi. In questa sezione si analizzano quattro strumenti fondamentali: sbt-crossproject, Scala.js, Scala Native e Cats, valutando le loro caratteristiche, vantaggi e possibili applicazioni avanzate.

3.2.1 sbt-crossproject

Sbt-crossproject è un plugin per sbt, acronimo di Simple Build Tool, che permette di gestire progetti multiplatforma in Scala, facilitando la condivisione di codice tra diverse piattaforme, come Java Virtual Machine, ambienti JavaScript e Scala Native. sbt-crossproject consente, se si sta sviluppando una libreria o un'applicazione che deve girare su più piattaforme, di definire un unico progetto con codice condiviso, separando solo le parti specifiche per ciascuna piattaforma.

Per utilizzare la libreria sarà sufficiente importarla e definire un progetto cross-platform come riportato nel listato 2.

```

1 enablePlugins(CrossProjectPlugin)
2 import sbtcrossproject.CrossProject
3 import sbtcrossproject.CrossPlugin.autoImport._
4
5 lazy val myProject =
6   crossProject(JSPlatform, JVMPlatform)
7     .in(file("myProject"))
8     .settings(
9       name := "my-crossproject",
10      version := "0.1.0"
11    )
12    .jsSettings(
13      scalaJSUseMainModuleInitializer := true
14    )
15    .jvmSettings(
16      // Impostazioni specifiche per la JVM
17    )
18
19 lazy val myProjectJVM = myProject.jvm
20 lazy val myProjectJS = myProject.js
21

```

Listing 2: Esempio d'uso: sbt-crossproject

Tra i vantaggi dell'impiego di questa libreria ci sono la possibilità di riutilizzare il codice e una maggiore agilità nello sviluppo di progetti full-stack. Se si lavora solamente con JVM e Scala.js, è anche possibile utilizzare Scala 3 Cross-Building, perciò questo plugin risulta utile in progetti di una certa complessità.

3.2.2 Scala.js

Scala.js consente di sfruttare congiuntamente gli ecosistemi Scala e JavaScript, permettendo agli sviluppatori di scrivere applicazioni robuste per browser utilizzando Scala.



Figura 3.2: Logo di ScalaJS

Uno degli aspetti più potenti di Scala.js è la possibilità di interagire direttamente con API JavaScript, grazie appunto alla sua stretta integrazione con questo ecosistema, mantenendo la sicurezza dei tipi.

Nel listato 3 è riportato un estratto di codice dove, considerando un caso in cui si debba interagire con l'API Fetch, si procede a recuperare dati da un servizio REST.

```

1 import scala.scalajs.js
2 import scala.scalajs.js.annotation._
3 import scala.concurrent.Future
4 import scala.scalajs.js.Promise
5 import scala.scalajs.concurrent.JSExecutionContext.Implicits.queue
6
7 @js.native
8 @JSGlobalScope
9 object FetchAPI extends js.Object {
10   def fetch(url: String): Promise[js.Dynamic] = js.native
11 }
12
13 def fetchData(url: String): Future[String] = {
14   FetchAPI.fetch(url).toFuture.map { response =>
15     response.selectDynamic("text").asInstanceOf[js.Function0[Promise[String]]]().toFuture
16   }
17 }

```



```

16     }.flatten
17   }
18
19   // Utilizzo
20   fetchData("https://api.example.com/data").foreach(println)

```

Listing 3: Esempio d'uso: Scala.js

In questo esempio:

- l'API Fetch viene invocata direttamente da Scala.js,
- la conversione da Promise a Future garantisce la compatibilità con la gestione asincrona di Scala,
- a funzione `selectDynamic` permette di accedere in modo sicuro ai campi dei JSON restituiti dall'API.

3.2.3 Scala Native

Scala Native è un compilatore *Ahead-Of-Time*, ovvero che trasforma il codice sorgente in codice macchina eseguibile prima dell'esecuzione del programma, al contrario dei compilatori *Just-in-Time*, e un runtime leggero con gestione automatizzata, progettato specificamente per Scala.[15]

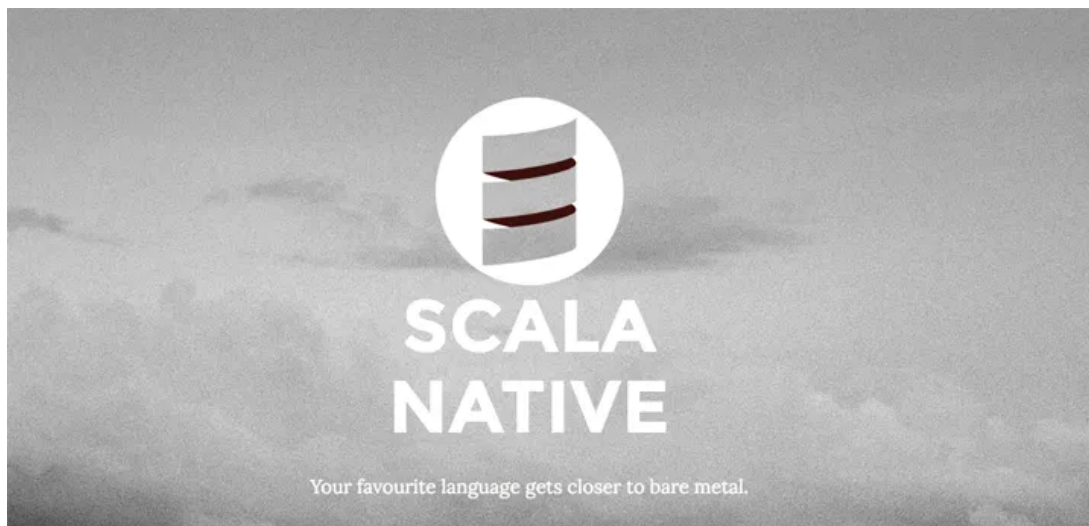


Figura 3.3: Logo e motto di Scala Native [3]

Esso consente di compilare codice Scala direttamente in codice di basso livello utilizzando l'infrastruttura LLVM, eliminando la dipendenza dalla JVM. Ciò elimina la fase di *warm-up* tipica dei compilatori Just-In-Time, garantendo tempi di avvio istantanei e prestazioni elevate. Una delle caratteristiche distintive di Scala Native è la disponibilità di primitive di basso livello, come puntatori e strutture, che permettono agli sviluppatori di ottimizzare manualmente le proprie applicazioni ottenendo così il massimo controllo sulle prestazioni e sull'utilizzo delle risorse. Scala Native offre inoltre un'interoperabilità senza soluzione di continuità con il codice nativo, consentendo chiamate dirette a librerie C e ad altre API a livello di sistema senza ulteriore overhead. Questa capacità è sfruttata attraverso l'uso di oggetti *extern*, che semplificano l'integrazione e l'interazione con il codice nativo.

Concludendo l'impiego di Scala Native risulta quindi particolarmente vantaggioso per applicazioni a bassa latenza, strumenti CLI e scenari *embedded*, garantendo elevate prestazioni e un'integrazione efficiente in ambienti nativi.

3.2.4 Cats

Cats, nome utilizzato come diminutivo di *category theory*, una teoria matematica che studia in modo astratto le strutture matematiche e le relazioni tra esse [16]. Questa libreria fornisce astrazioni per la programmazione funzionale nel linguaggio Scala, ed è disponibile anche per Scala.js e Scala Native, oltre al runtime JVM standard.

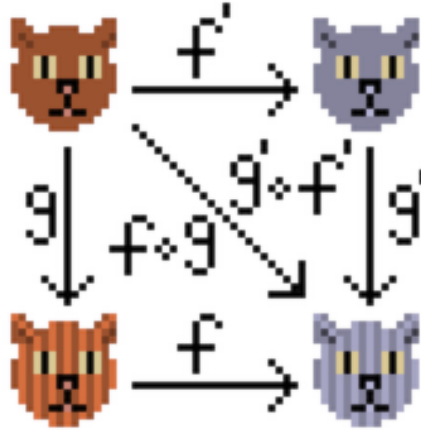


Figura 3.4: Logo Cats [4]

Essa si propone di fornire astrazioni per la programmazione funzionale che siano compatibili a livello binario, modulari, accessibili ed efficienti. Un obiettivo più ampio è quello di costituire una base solida per un ecosistema di librerie puramente funzionali e fortemente tipizzate, a supporto dello sviluppo di applicazioni Scala basate sui principi della programmazione funzionale.

Tra le principali caratteristiche di Cats si possono trovare:

- **Type Classes:** strutture che rappresentano concetti fondamentali nella programmazione funzionale. Alcune delle più rilevanti includono:
 - **Functor:** astrazione che rappresenta mappature su contenitori o contesti.
 - **Applicative:** estende i Functor permettendo l'applicazione di funzioni incapsulate in un contesto a valori anch'essi in un contesto.
 - **Monad:** astrazione che consente la composizione di operazioni sequenziali su contesti.
 - **Foldable:** consente di ridurre strutture dati a un singolo valore.
 - **Traverse:** combina le funzionalità di *mapping* e *folding* su strutture dati.
- **Data Types:** Cats fornisce diversi tipi di dati utili per la programmazione funzionale, tra cui:
 - **Either:** rappresenta un valore che può essere di due tipi, comunemente utilizzato per gestire errori o risultati alternativi.
 - **Validated:** simile a Either, ma più adatto per la validazione che accumula errori.
 - **NonEmptyList:** una lista che garantisce la presenza di almeno un elemento, utile per evitare operazioni su liste vuote.
 - **Kleisli:** rappresenta una funzione che restituisce un valore in un contesto monadico, utile per la composizione di funzioni che producono effetti.

3.2.4.1 Cats Effect

Cats Effect è un modulo di Cats che fornisce astrazioni per gestire effetti collaterali in modo puro e funzionale. Introduce il tipo di dato IO per rappresentare computazioni che possono eseguire effetti prima di restituire un valore. IO è una struttura dati pura e immutabile che preserva la trasparenza referenziale, rendendola adatta alla programmazione funzionale. Essa offre strumenti per la gestione sicura delle

risorse, garantendo che siano allocate e rilasciate correttamente anche in presenza di eccezioni o cancellazioni. Il tipo `Resource` consente invece di gestire l'acquisizione e il rilascio di risorse in modo sicuro e compositivo. In più, `Cats Effect` fornisce primitive per la programmazione concorrente, come `Fiber`, che rappresenta un'unità leggera di esecuzione concorrente, e strutture come `Deferred`, `Ref` e `Semaphore` per coordinare le operazioni concorrenti.

Capitolo 4

Applicazione e test

In questo capitolo verranno illustrate alcune demo effettuate allo scopo di esplorare alcune delle tecnologie viste nei capitoli precedenti e analizzare i benefici e i vincoli di ogni soluzione.

4.1 SqueezeNet

SqueezeNet è una rete neurale profonda rilasciata nel 2016, che si occupa di classificazione di immagini. Si tratta di una rete neurale convoluzionale, composta da 18 strati e progettata per ottenere elevate prestazioni con un numero ridotto di parametri, mantenendo un'accuratezza comparabile a quella di reti più complesse come AlexNet. Per utilizzarla, è possibile caricare una versione pre-addestrata su oltre un milione di immagini (provenienti dal database ImageNet) della rete. Questa rete è particolarmente utile in applicazioni dove è necessario un bilanciamento tra accuratezza e leggerezza del modello, come l'impiego su dispositivi a risorse limitate.

Per questa ricerca, sono state effettuate delle demo che adottano SqueezeNet, per diverse ragioni principalmente legate alla sua diffusione, efficienza e semplicità d'uso.

4.2 Demo

Per quanto riguarda la parte applicativa di questo progetto, si è preferito seguire un approccio esplorativo analogo a quello seguito nella sezione teorica, senza sviluppare complessi sistemi, ma concentrandosi sull'approfondimento di una semplice implementazione nelle sue possibili forme, analizzando analogie e differenze.

4.2.1 Full-model API

Alla base dello sviluppo, prendendo spunto dall'idea utilizzata per presentare la libreria, si trova l'utilizzo di un modello per il riconoscimento di immagini per effettuare un'inferenza. Per effettuare questa analisi sono state utilizzate tre metodologie diverse:

- implementazione in Python sfruttando Google Colab
- implementazione in Scala
- implementazione in Scala utilizzando la libreria ONNX-Scala

I parametri fissati per queste tre implementazioni sono stati `squeezenet1.1-7.onnx`¹ come modello utilizzato, il suo corrispettivo file delle etichette `synset.txt`² e l'immagine utilizzata per i test riportata nella figura

4.1³

¹<https://github.com/onnx/models/raw/main/validated/vision/classification/squeezenet/model/squeezenet1.1-7.onnx>

²<https://s3.amazonaws.com/onnx-model-zoo/synset.txt>

³<https://s3.amazonaws.com/model-server/inputs/kitten.jpg>



Figura 4.1: Foto di gatto

L'implementazione in Python ha richiesto innanzitutto l'impostazione dell'ambiente e delle librerie in quanto Google Colab collega un nuovo runtime a ogni avvio. Dopo aver verificato la corretta versione delle librerie installate, vengono definite le funzioni e infine richiamata l'inferenza. Il codice riportato qui di seguito avrà l'ordine inverso per una maggior facilità di lettura. Nel codice riportato nel listato 4 si può osservare la chiamata e la definizione della funzione principale per definire la classe di appartenenza dell'immagine. La funzione si suddivide in tre sezioni principali, *preprocessing*, *inferenza* e *postprocessing*.

```
1  img_path = 'kitten.jpg'
2  predict(img_path)
3
4  def predict(path):
5      img = get_image(path, show=True)
6      img = preprocess(img)
7      mod.forward(Batch([img]))
8      # Take softmax to generate probabilities
9      scores = mx.ndarray.softmax(mod.get_outputs()[0]).asnumpy()
10     # print the top-5 inferences class
11     scores = np.squeeze(scores)
12     a = np.argsort(scores)[::-1]
13     for i in a[0:5]:
14         print('%f class=%s ; probability=%f' %(i, labels[i], scores[i]))
```

Listing 4: Python: Funzione di inferenza

Nel listato 5 si può osservare la principale funzione di preprocessing, la quale prende in ingresso l'immagine caricata dalla directory indicata dal path impostato, le applica la funzione `transform_fn` che la trasforma in un tensore normalizzato rappresentante l'immagine di dimensione 224 per 224 pixel che a sua volta verrà poi passato al modello per effettuare l'inferenza con l'istruzione `mod.forward(Batch([img]))`.

```
1  def preprocess(img):
2      transform_fn = transforms.Compose([
3          transforms.Resize(256),
4          transforms.CenterCrop(224),
5          transforms.ToTensor(),
6          transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
7      ])
8      img = transform_fn(img)
9      img = img.expand_dims(axis=0)
10     return img
```

Listing 5: Python: Preprocessing

A seguire vengono effettuate le operazioni di postprocessing necessarie per ottenere una versione leggibile dei risultati del modello. Ordinando le probabilità ottenute e incrociandole con le informazioni presenti nel file `synset.txt` il risultato sarà quello riportato nel listato 6, dove è possibile leggere il codice dell'etichetta, l'oggetto rappresentato e la probabilità con la quale è stato identificato.

```

1 285.000000 class=n02124075 Egyptian cat ; probability=0.416662
2 282.000000 class=n02123159 tiger cat ; probability=0.204020
3 281.000000 class=n02123045 tabby, tabby cat ; probability=0.195221
4 287.000000 class=n02127052 lynx, catamount ; probability=0.093994
5 286.000000 class=n02125311 cougar, puma, catamount, mountain lion, painter, panther, Felis
  ↪ concolor ; probability=0.088270

```

Listing 6: Python: Risultato postprocessing

Le due implementazioni in Scala condividono alcune parti, come la funzione di postprocessing che converte da probabilità a etichette testuali, o la funzione di preprocessing riportata nel listato 7. Questa, dopo aver importato l'immagine, procederà a ridimensionarla per aderire al formato atteso, per poi convertirla in un Array.

```

1 def loadAndTransform(imagePath: String, width: Int = 224, height: Int = 224): Array[Float] = {
2   val img: BufferedImage = ImageIO.read(new File(imagePath))
3
4   // Resize image
5   val resized = img.getScaledInstance(width, height, java.awt.Image.SCALE_SMOOTH)
6   val buffered = new BufferedImage(width, height, BufferedImage.TYPE_3BYTE_BGR)
7   val g = buffered.getGraphics
8   g.drawImage(resized, 0, 0, null)
9   g.dispose()
10
11  // Convert to float array and normalize
12  val raster = buffered.getRaster
13  val pixels = raster.getDataBuffer.asInstanceOf[java.awt.image.DataBufferByte].getData
14  val floatPixels = new Array[Float](3 * width * height)
15
16  // SqueezeNet expects input in (C, H, W) order not (HWC)
17  for (y <- 0 until height; x <- 0 until width) {
18    val i = (y * width + x) * 3 // HWC index
19    val r = (pixels(i) & 0xFF) / 255.0f
20    val g = (pixels(i + 1) & 0xFF) / 255.0f
21    val b = (pixels(i + 2) & 0xFF) / 255.0f
22
23    // Normalize using ImageNet mean & std
24    val mean = Array(0.485f, 0.456f, 0.406f)
25    val std = Array(0.229f, 0.224f, 0.225f)
26
27    // Store in CHW format
28    floatPixels(y * width + x) = (r - mean(0)) / std(0) // Red
29    floatPixels(width * height + y * width + x) = (g - mean(1)) / std(1) // Green
30    floatPixels(2 * width * height + y * width + x) = (b - mean(2)) / std(2) // Blue
31  }
32  floatPixels
33 }

```

Listing 7: Scala: Preprocessing

Esistono due formati principali utilizzati per determinare la forma di un tensore nel mondo del Deep Learning, e in particolare in quello delle *Convolutional Neural Networks* (CNNs). Questi formati determinano come i dati multidimensionali, come immagini, nuvole di punti o feature map, vengano salvati in memoria.

- **NHWC**: indicante *Number of samples, Height, Width, Channels*, questo formato memorizza i dati in maniera che, per ogni oggetto, in questo caso immagine, altezza e larghezza vengano prima, seguite dai diversi canali. Il formato NHWC è comunemente associato a TensorFlow.
- **NCHW**: indicante *Number of samples, Channels, Height, Width*, questo formato memorizza i dati in maniera che, per ogni oggetto, in questo caso immagine, i canali precedano le dimensioni di altezza e larghezza. Il formato NCHW è comunemente associato a PyTorch.

La scelta tra questi due formati può impattare in maniera importante sugli accessi in memoria, sull'efficienza computazionale e soprattutto sulla compatibilità con diversi framework. In questo caso, come detto in precedenza, SqueezeNet utilizza il formato NCHW, per cui è stato necessario, durante la normalizzazione, disporre i valori estratti dall'Array all'interno del tensore applicando una logica *channel first*.

Nel listato 8 è riportato un estratto dei punti fondamentali nel codice sviluppato per l'inferenza in Scala. Come possibile vedere fin dalle prime righe è stata utilizzata la libreria ONNX Runtime per avere accesso al modello. Una volta ottenuta la versione dell'immagine sotto forma di tensore, questa verrà unita a un Array contenente i valori che ne definiscono la sua *shape*. Questo nuovo tensore verrà poi passato al modello per estrarre il risultato finale.

```
1 import ai.onnxruntime.OrtSession.Result
2 val env = OrtEnvironment.getEnvironment
3 val session = env.createSession("res/squeezenet1.1-7.onnx", new OrtSession.SessionOptions())
4
5 val imageTensor = ImageUtils.loadAndTransform(imagePath)
6
7 val shape = Array(1L, 3L, 224L, 224L) // Ensure correct input shape
8 val inputs = Map(inputName -> OnnxTensor.createTensor(env,
9   ↪ java.nio.FloatBuffer.wrap(imageTensor), shape))
10
11 val results = session.run(inputs.asJava)
```

Listing 8: Scala: Inferenza

Nel listato 9 è riportato invece un estratto dei punti fondamentali nel codice sviluppato per l'inferenza utilizzando la libreria ONNX-Scala. Viene ora utilizzato per l'accesso al modello un backend diverso, messo a disposizione dalla libreria stessa. Analogamente alla versione precedente anche in questa viene processata l'immagine e associata alla forma del tensore, che però può accettare in ingresso anche le variabili `tensorDenotation` e `tensorShapeDenotation` utilizzabili per denotare le dimensioni indicate. Una volta creato il tensore sarà poi sufficiente passarlo in ingresso al modello per ottenere il risultato.

```
1 import org.emergentorder.onnx.backends.ORTModelBackend
2 val modelPath = "res/squeezenet1.1-7.onnx"
3 val squeezenetBytes = Files.readAllBytes(Paths.get("res/squeezenet1.1-7.onnx"))
4 val model = new ORTModelBackend(squeezenetBytes)
5
6 val image = ImageUtils.loadAndTransform(imagePath)
7
8 //In NCHW tensor image format
9 val shape = 1 #: 3 #: 224 #: 224 #: SNil
10 val tensorShapeDenotation = "Batch" ##: "Channel" ##: "Height" ##: "Width" ##: TSNil
11 val tensorDenotation: String & Singleton = "Image"
12
13 val imageTens = Tensor(image,tensorDenotation,tensorShapeDenotation,shape)
14 //or as a shorthand if you aren't concerned with enforcing denotations
15 val imageTensDefaultDenotations = Tensor(image,shape)
16
17 val outputs = model.fullModel[Float,
18     "ImageNetClassification",
19     "Batch" ##: "Class" ##: TSNil,
20     1 #: 1000 #: SNil] (Tuple(imageTens))
21
```

Listing 9: ONNX-Scala: Inferenza

4.2.2 Fine-grained API

Nella figura 4.2 si può osservare un esempio semplicistico di rete neurale di tipo *feed-forward*, la quale applica una serie di trasformazioni a un tensore in ingresso per generare un output. Nel listato 10 è riportata a scopo dimostrativo l'implementazione della stessa rete utilizzando le API fornite da ONNX-Scala utilizzando le funzioni di libreria equivalenti.

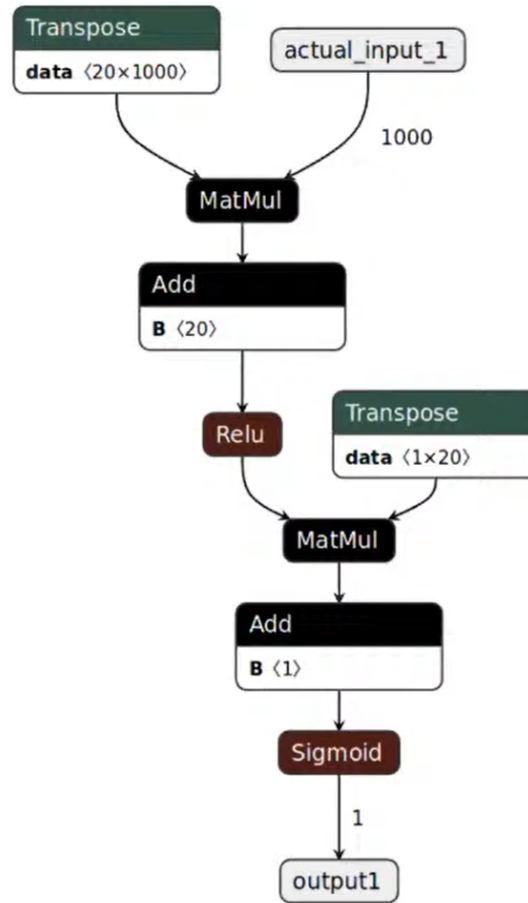


Figura 4.2: Rete neurale feed-forward

```
1 class FFnn(byteArray: Array[Byte]) {
2   val backend = ORTOperatorBackendAll()
3   val bytesDataSource = ONNXBytesDataSource(byteArray)
4
5   def program(input: Tensor[Float, (TT, TD, InputShape)]) = {
6     val nodeLearned0 = dataSource.getParams[Float]("learned_0")
7     val nodeLearned1 = dataSource.getParams[Float]("learned_1")
8     val nodeLearned2 = dataSource.getParams[Float]("learned_2")
9     val nodeLearned3 = dataSource.getParams[Float]("learned_3")
10    val node5 = backend.TransposeV1("5", perm = Some(Array(1, 0)), data = nodeLearned0)
11    val node6 = backend.MatMulV9("6", A = input, B = node5)
12    val node7 = backend.AddV7("7", A = node6, B = nodeLearned1)
13    val node8 = backend.ReluV6("8", x = node7)
14    val node9 = backend.TransposeV1("9", perm = Some(Array(1, 0)), data = nodeLearned2)
15    val node10 = backend.MatMulV9("10", A = node8, B = node9)
16    val node11 = backend.AddV7("11", A = node10, B = nodeLearned3)
17    val output = backend.SigmoidV6("output", x = node11)
18    output
19  }
20 }
```

Listing 10: Rete neurale feed-forward

Il punto focale di queste API risulta essere proprio la Type Safety, perciò nel listato 11 è riportato il codice utilizzato per verificare la capacità della libreria di introdurre la sicurezza prevista. La definizione della funzione SqrtV13 prevede che il tensore in ingresso contenga valori di tipo BFloat16, Float16, Float o Double, per cui il tensore longTens, composto da valori di tipo Long causerebbe problemi in fase di esecuzione nel caso fosse impostato come input. Come atteso il compilatore riesce a riconoscere l'errore preventivamente, impedendo l'esecuzione e segnalando la riga 10 come errata, a differenza delle istruzioni alle righe 9 e 11.

```

1  object TypeSafety {
2      val onnxBackend = new ORTOperatorBackendAll()
3      val shape =          1      #:      3      #:      224      #: 224      #: SNil
4      val tensorShapeDenotation = "Batch" ##: "Channel" ##: "Height" ##: "Width" ##: TSNil
5      val tensorDenotation: String & Singleton = "Image"
6
7      val longTens =
8          ↪ Tensor(Array.fill(1*3*224*224){-42l}, tensorDenotation, tensorShapeDenotation, shape)
9      val floatTens =
10         ↪ Tensor(Array.fill(1*3*224*224){42f}, tensorDenotation, tensorShapeDenotation, shape)
11     onnxBackend.AbsV13("abs", longTens)
12     onnxBackend.SqrtV13("sqrt", longTens) // ERROR does not compile
13     onnxBackend.SqrtV13("sqrt", floatTens)
14 }

```

Listing 11: Scala: Fine-grained API

Capitolo 5

Risultati e conclusioni

La sezione dedicata ai risultati presenta un'analisi dettagliata delle prestazioni dei modelli testati, evidenziando i principali vantaggi e le eventuali aree di miglioramento. I test condotti dimostrano la versatilità di ONNX-Scala nell'integrare modelli complessi in diversi scenari applicativi.

5.1 Benchmark

Utilizzando la libreria JUnit sono poi stati effettuati alcuni test sui tempi di esecuzione osservati durante la fase di sviluppo e i risultati sono stati confermati. In corrispondenza di una singola inferenza la funzione che utilizza la libreria ONNX-Scala è risultata essere più lenta rispetto al suo corrispettivo senza intermediari, tuttavia aumentando il numero di inferenze si è osservata un'inversione nella tendenza, avendo tempi di esecuzione molto più rapidi nel primo caso. Da sottolineare che i test sono stati svolti solo su una ripetizione della chiamata alla funzione che effettua l'inferenza, senza includere i processi di caricamento del modello, di preprocessing dell'immagine e di postprocessing dei risultati. Nella figura 5.1 è poi possibile vedere i tempi risultanti dai test.

```
1  @Test def testSingleExecution() =
2    val args = Array(path)
3    val library_time = TypeSafeONNXInference.main(args)
4    val raw_time = RawONNXInference.main(args)
5    System.out.println("ONNX-Scala: " + library_time)
6    System.out.println("ONNX: " + raw_time)
7    assertTrue(library_time > raw_time)
8
9  @Test def testBatchExecution() =
10    val args = Array(path, "2000")
11    val library_time = TypeSafeONNXInference.main(args)
12    val raw_time = RawONNXInference.main(args)
13    System.out.println("ONNX-Scala: " + library_time)
14    System.out.println("ONNX: " + raw_time)
15    assertTrue(library_time < raw_time)
```

Listing 12: Codice JUnit per l'esecuzione dei test

```
ONNX-scala: 0.5016682
ONNX: 0.0164208
ONNX-scala: 0.2620282
ONNX: 7.9952702
Execution took 10s
2 tests, 2 passed
All tests in onnx.OnnxTest passed

=====
Total duration: 10s
All 1 test suites passed.
=====
```

Figura 5.1: Risultati dei benchmark

5.2 Conclusioni

Questa ricerca ha dimostrato come l'integrazione tra il linguaggio Scala e il formato ONNX possa offrire nuove opportunità nel campo del Machine Learning, combinando la sicurezza dei tipi con la flessibilità e l'interoperabilità dei modelli. L'utilizzo di ONNX-Scala consente di sfruttare i vantaggi del formato ONNX all'interno dell'ecosistema Scala, offrendo agli sviluppatori un ambiente altamente tipizzato e funzionale per costruire pipeline di inferenza robuste e scalabili.

Per quanto riguarda la Full-model API i risultati dei test condotti evidenziano chiaramente che ONNX-Scala rappresenti una soluzione promettente per scenari applicativi complessi. Sebbene l'approccio basato su Scala presenti un overhead maggiore rispetto a Python in alcuni casi, l'efficienza migliora sensibilmente con l'applicazione su domini più adatti, soprattutto grazie all'integrazione con ONNX Runtime. La Fine-grained API di ONNX-Scala, seppur più complessa da utilizzare, garantisce un controllo approfondito sui modelli e sulla gestione dei tensori, riducendo significativamente il rischio di errori a runtime.

Partendo dalla libreria ONNX-Scala, punto centrale di questo progetto di ricerca, si è seguito un approccio esplorativo al fine di raccogliere in questo elaborato tutte le conoscenze trasversali necessarie per una sua miglior comprensione e applicazione. Al termine di questo progetto è stato possibile riconoscere i vantaggi nell'impiego di questa libreria in possibili progetti futuri con le dovute limitazioni.

Per quanto riguarda l'applicazione su modelli già addestrati utilizzati come *black-box* la libreria è risultata essere utile ma non fondamentale, in quanto le API Full-model non riescono a garantire una Type Safety sufficientemente presente, ma forniscono comunque un modo rapido di effettuare inferenze su grafi ottimizzati all'interno dell'ambiente Scala.

Per quanto riguarda applicazioni di più basso livello, implementate invece utilizzando le API Fine-grained, si ha un vantaggio maggiore nell'utilizzare la libreria sotto esame poiché permettendo di creare e modificare modelli di diverse dimensioni, crea la possibilità di avere una sorta di *sandbox* all'interno della quale poter sfruttare maggiormente la tipizzazione di Scala per la definizione di grafi ONNX, che grazie alla loro natura si sposano efficacemente con la programmazione funzionale.

Riferimenti bibliografici

- [1] Michael Hicks. What is type safety? - The PL Enthusiast — pl-enthusiast.net. <http://www.pl-enthusiast.net/2014/08/05/type-safety/>. [Accessed 28-01-2025].
- [2] What is ONNX? — microsoft.github.io. <https://microsoft.github.io/ai-at-edge/docs/onnx/>. [Accessed 25-01-2025].
- [3] SE Daily. Scala Native with Denys Shabalin - Software Engineering Daily — softwareengineeringdaily.com. <https://softwareengineeringdaily.com/2017/10/16/scala-native-with-denys-shabalin/>. [Accessed 31-01-2025].
- [4] Kailuo Wang. Cats — typelevel.org. <https://typelevel.org/cats/>. [Accessed 02-02-2025].
- [5] ONNX | Home — onnx.ai. <https://onnx.ai/>. [Accessed 24-01-2025].
- [6] Protocol Buffers — protobuf.dev. <https://protobuf.dev/>. [Accessed 26-01-2025].
- [7] onnx/docs/Operators.md at main · onnx/onnx — github.com. <https://github.com/onnx/onnx/blob/main/docs/Operators.md>. [Accessed 02-02-2025].
- [8] onnx/docs/TypeDenotation.md at main · onnx/onnx — github.com. <https://github.com/onnx/onnx/blob/main/docs/TypeDenotation.md>. [Accessed 01-02-2025].
- [9] onnx/docs/DimensionDenotation.md at main · onnx/onnx — github.com. <https://github.com/onnx/onnx/blob/main/docs/DimensionDenotation.md>. [Accessed 01-02-2025].
- [10] GitHub - EmergentOrder/onnx-scala: An ONNX (Open Neural Network eXchange) API and backend for typeful, functional deep learning and classical machine learning in Scala 3 — github.com. <https://github.com/EmergentOrder/onnx-scala>. [Accessed 24-01-2025].
- [11] onnxruntime/js/web at main · microsoft/onnxruntime — github.com. <https://github.com/microsoft/onnxruntime/tree/main/js/web>. [Accessed 02-02-2025].
- [12] YouTube. https://www.youtube.com/watch?v=HyYpMJNVk&ab_channel=FunctionalTV. [Accessed 23-01-2025].
- [13] GitHub - mandubian/neurocat: From neural networks to the Category of composable supervised learning algorithms in Scala with compile-time matrix checking based on singleton-types — github.com. <https://github.com/mandubian/neurocat>. [Accessed 27-01-2025].
- [14] Shadaj Laddad and Koushik Sen. Scalapy: seamless python interoperability for cross-platform scala programs. In *Proceedings of the 11th ACM SIGPLAN International Symposium on Scala*, pages 2–13, 2020.
- [15] Scala Native &x2014; Scala Native 0.5.6 documentation — scala-native.org. <https://scala-native.org/en/stable/>. [Accessed 31-01-2025].
- [16] Category theory - Wikipedia — en.wikipedia.org. https://en.wikipedia.org/wiki/Category_theory. [Accessed 31-01-2025].
- [17] Ultralytics. ONNX — docs.ultralytics.com. <https://docs.ultralytics.com/it/integrations/onnx/>. [Accessed 26-01-2025].
- [18] Hong Zhang and Haijian Shao. Exploring the latest applications of openai and chatgpt: An in-depth survey. *CMES-Computer Modeling in Engineering & Sciences*, 138(3), 2024.
- [19] YouTube. <https://www.youtube.com/watch?v=nRlnSy4Vbnc&t=778s>. [Accessed 23-01-2025].
- [20] YouTube. <https://www.youtube.com/watch?v=U5kaWtPINcI>. [Accessed 23-01-2025].
- [21] YouTube. <https://www.youtube.com/watch?v=8HuZTeHi7lg&t=1156s>. [Accessed 23-01-2025].

- [22] SqueezeNet - Wikipedia — en.wikipedia.org. <https://en.wikipedia.org/wiki/SqueezeNet>. [Accessed 23-01-2025].
- [23] GitHub - cric96/scala-machine-learning-libraries — github.com. <https://github.com/cric96/scala-machine-learning-libraries/tree/main>. [Accessed 23-01-2025].
- [24] GitHub - MaximeKjaer/tf-dotty: Shape-safe TensorFlow in Dotty — github.com. <https://github.com/MaximeKjaer/tf-dotty/tree/master>. [Accessed 23-01-2025].
- [25] GitHub - portable-scala/sbt-crossproject: Cross-platform compilation support for sbt. — github.com. <https://github.com/portable-scala/sbt-crossproject>. [Accessed 23-01-2025].
- [26] Scala.js — scala-js.org. <https://www.scala-js.org/>. [Accessed 23-01-2025].
- [27] 262588213843476. text: imagenet 1000 class idx to human readable labels (Fox, E., & Guestrin, C. (n.d.). Coursera Machine Learning Specialization.) — gist.github.com. <https://gist.github.com/yrevar/942d3a0ac09ec9e5eb3a>. [Accessed 23-01-2025].
- [28] SqueezeNet — pytorch.org. https://pytorch.org/hub/pytorch_vision_squeezenet/. [Accessed 23-01-2025].